

Operating Systems: Worksheet 1

This project shows the basics of 32-bit x86 assembly programming.

I've written several programs that work directly with CPU registers and memory, interfaced them with C code, and automated everything with a Makefile.

How to Run the Code

I've set up a Makefile to make building and running the code super easy.

1. **Clean Start:** First, clear out any old files to ensure a fresh build.
`make clean`
2. **Build Everything:** Compile all the programs at once.
`make`
3. **Run the Programs:** You can run each task individually from the terminal:
`./task1` # Simple addition
`./task1.2` # Interactive addition
`./task2.1` # Name and welcome loop
`./task2.2` # Sum numbers 1-100
`./task2.3` # Sum a specific range



Task Breakdown

Here is what I built for each task, how it works, and proof that it runs successfully.

♦ Task 1: Getting Started

The goal here was to get comfortable with the basics: moving data into registers, doing math, and printing the results.

Task 1.1: Simple Addition

This program takes two hard-coded numbers (15 and 6), adds them together, and prints the result.

How it works:

It loads the numbers from memory into the EAX register, adds them up, and then calls `print_int` to show the answer.

; --- Code Snippet from task1.asm ---

mov eax, [num1] ; Load 15 into EAX

add eax, [num2] ; Add 6 to EAX (15 + 6 = 21)

call print_string ; Print the result (21)

```
C driver.c task1.asm COMMIT_EDITMSG README.md Makefile task1.2.o
src > ASM task1.asm
1  %include "asm_io.inc"
2  ;notes
3  segment .data
4      ;two integers to add
5      num1    dd  15          ; First number (15)
6      num2    dd  6           ; Second number (6)
7      result  dd  0           ; Storage for result
8      msg1    db  "The sum is: ", 0
9
10 segment .bss
11     ; No uninitialized data needed
12
13 segment .text
14     global asm_main
15
16 asm_main:
17     enter    0,0             ; Setup stack frame
18     pusha    ; Save all registers
19
20     ; Print message
21     mov      eax, msg1
22     call     print_string
23
24     ; Load first number into EAX storage
25     mov      eax, [num1]     ; EAX = num1 (15)
26
27     ; Add second number to EAX storage
28     add      eax, [num2]     ; EAX = EAX + num2 (15 + 6 = 21)
29
```

Output:

```
/usr/bin/ld: NOTE: This behaviour is deprecated and will be removed in a future version of
● immanuel12.mohammed@live.uwe.ac.uk@csctcloud:~/os/OS-WKSHEET-1$ ./task1
The sum is: 21
```

Task 1.2: Interactive Calculator

This version is more dynamic; it prompts the user for the numbers, adds them, and prints a formatted sentence.

How it works:

I used the `read_int` function to get input from the keyboard and stored those values in reserved memory slots (`resd`).

; --- Code Snippet from task1.2.asm ---

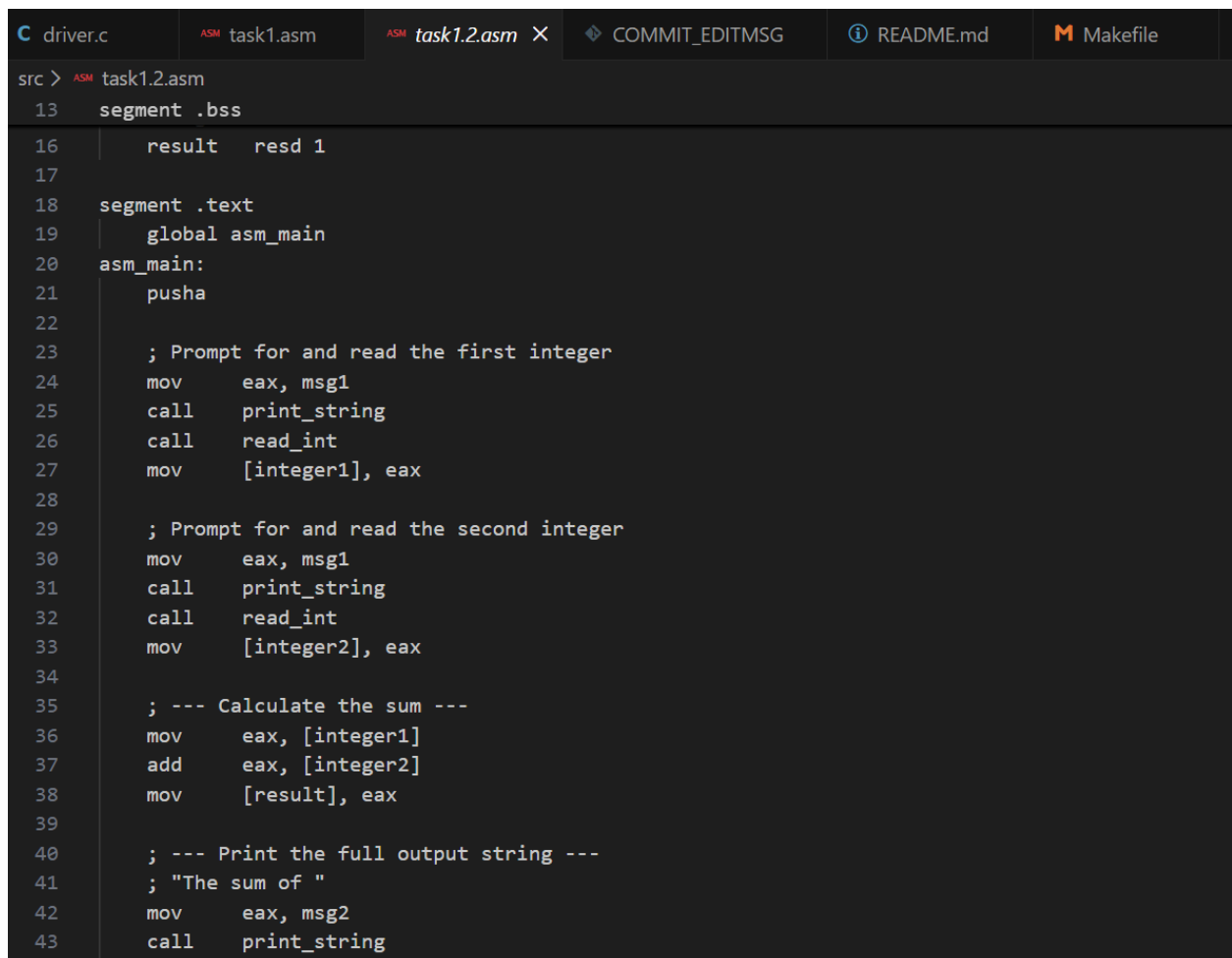
`call read_int` ; Wait for user input

`mov [integer1], eax` ; Save the first number

...

`add eax, [integer1]` ; Add the first number to the second

`call print_int` ; Print the final sum

A screenshot of a code editor with a dark theme. The editor has several tabs at the top: 'driver.c', 'ASM task1.asm', 'ASM task1.2.asm' (which is the active tab), 'COMMIT_EDITMSG', 'README.md', and 'Makefile'. The main area shows assembly code for 'task1.2.asm'. The code starts with a segment declaration for '.bss' containing a 'result' variable of type 'resd 1'. Then, a '.text' segment is declared, containing a 'global asm_main' directive and the 'asm_main' function. The function begins with a 'pusha' instruction. It then prompts for and reads the first integer using 'mov eax, msg1', 'call print_string', 'call read_int', and 'mov [integer1], eax'. This sequence is repeated for the second integer. Following this, a comment indicates the calculation of the sum: '; --- Calculate the sum ---'. The code then performs 'mov eax, [integer1]', 'add eax, [integer2]', and 'mov [result], eax'. Finally, another comment indicates printing the full output string: '; --- Print the full output string ---', followed by 'mov eax, msg2' and 'call print_string'. Line numbers 13 through 43 are visible on the left side of the code block.

```
src > ASM task1.2.asm
13  segment .bss
14
15      result    resd 1
16
17
18  segment .text
19      global asm_main
20  asm_main:
21      pusha
22
23      ; Prompt for and read the first integer
24      mov     eax, msg1
25      call    print_string
26      call    read_int
27      mov     [integer1], eax
28
29      ; Prompt for and read the second integer
30      mov     eax, msg1
31      call    print_string
32      call    read_int
33      mov     [integer2], eax
34
35      ; --- Calculate the sum ---
36      mov     eax, [integer1]
37      add     eax, [integer2]
38      mov     [result], eax
39
40      ; --- Print the full output string ---
41      ; "The sum of "
42      mov     eax, msg2
43      call    print_string
```

Output:

```
immanuel12.mohammed@live.uwe.ac.uk@csctcloud:~/os/OS-WKSHEET-1$ ./task1.2
Enter a number: 2
Enter a number: 3
The sum of 2 and 3 is: 5
```

◆ Task 2: Loops and Logic

This task was about controlling the flow of the program using loops and if/else logic (conditional jumps).

Task 2.1: The Welcome Loop

This program asks for your name and a number (between 51-99). It then says "Welcome" to you that many times.

How it works:

- **Reading a String:** The provided library didn't have a "read string" function, so I had to write my own loop. It reads one character at a time until you hit Enter.
- **Validation:** It checks if your number is valid (51-99) using cmp (compare). If it's too high or too low, it jumps to an error message.
- **The Loop:** It uses the loop instruction, which uses the ECX register as a countdown timer.

; --- Code Snippet from task2.1.asm ---

```
.read_name_loop:      ; My custom loop to read a name
    call read_char
    cmp  eax, 10      ; Is it the Enter key?
    je   .done_reading
    mov  [edi], al     ; Save the character
    inc  edi           ; Move to next spot in memory
    jmp  .read_name_loop
```

Output:


```
immanuel2.mohammed@live.uwe.ac.uk@csctcloud:~/os/OS-WKSHEET-1$ ./task2.2
Initializing array with values 1-100...
Sum of array: 5050
```

Task 2.3: Summing a Range

This extends the previous program. You get to choose the start and end of the sum (e.g., "sum numbers from 5 to 10").

How it works:

The tricky part here was calculating the correct starting memory address. I used the LEA (Load Effective Address) instruction to figure out exactly where in memory the summation should begin based on your input.

Output:

```
immanuel2.mohammed@live.uwe.ac.uk@csctcloud:~/os/OS-WKSHEET-1$ ./task2.3
Array initialized with values 1-100
Enter start index (0-99): 1
Enter end index (0-99): 30
Sum of elements from index 1 to 30 is: 495
```

♦ Task 3: Automating with Make

To avoid typing long compilation commands every time, I created a Makefile. This script automates the entire build process.

Challenges & Fixes:

I ran into a few bugs while setting this up, which was a great learning experience:

1. **The "Missing File" Error:** The assembler couldn't find `asm_io.inc`.
 - *Fix:* I had to tell the assembler (`nasm`) to look inside the `src/` folder by adding the `-Isrc/` flag.
2. **The "Underscore" Mix-up:** My files were named `task2_2.asm`, but the Makefile expected `task2.2.asm`.
 - *Fix:* I renamed the files to be consistent with dots (e.g., `task2.2.asm`).
3. **The "Clean" Issue:** `make clean` wasn't deleting the old files inside the `src/` folder.
 - *Fix:* I updated the clean command to remove `src/*.o` files too.